# Librería API

Documentación del proyecto

## 1. Repositorio de GitHub

Enlace al repositorio con el código fuente del proyecto:

https://github.com/Emma67g/LibreriaAPI.git

## 2. Descripción de las APIs

El proyecto es una aplicación que contiene tres APIs REST para administrar una librería: Libros, Usuarios y Préstamos. Está organizada en capas (controllers, services, repositories y models) y guarda la información en MySQL.

Todas las APIs devuelven los errores en formato JSON ({"error": "mensaje"}) con los códigos 400 (datos inválidos), 404 (recurso inexistente) y 409 (regla de negocio o dato duplicado).

### 2.1 API de Libros

Administra el catálogo de la librería. Cada libro guarda su ISBN (10 o 13 dígitos, único), título, autor, editorial, año de publicación y número de ejemplares disponibles. Valida los datos de entrada y no permite eliminar libros que tengan préstamos asociados. En la ruta, `{isbn}` es el identificador interno del libro; el ISBN real se guarda en el campo `codigoIsbn`.

| Método | Ruta | Descripción |
|---|---|---|
| GET | `/api/libros` | Lista todos los libros |
| GET | `/api/libros/{isbn}` | Consulta un libro |
| POST | `/api/libros` | Registra un libro |
| PUT | `/api/libros/{isbn}` | Actualiza un libro |
| DELETE | `/api/libros/{isbn}` | Elimina un libro (si no tiene préstamos) |

### 2.2 API de Usuarios

Administra a los socios de la librería: nombre, apellidos, correo electrónico (único), contraseña, rol (ADMIN o USUARIO), estado (ACTIVO o INACTIVO), fecha de nacimiento, teléfono y dirección. La contraseña se recibe al registrar o editar, pero nunca se devuelve en las respuestas. No se puede eliminar un usuario que tenga préstamos asociados.

| Método | Ruta | Descripción |
|---|---|---|
| GET | `/api/usuarios` | Lista todos los usuarios |
| GET | `/api/usuarios/{id}` | Consulta un usuario |
| POST | `/api/usuarios` | Registra un usuario |

| Método | Ruta | Descripción |
|---|---|---|
| PUT | `/api/usuarios/{id}` | Actualiza un usuario |
| DELETE | `/api/usuarios/{id}` | Elimina un usuario (si no tiene préstamos) |

### 2.3 API de Préstamos

Registra el préstamo de un libro a un socio y su devolución, calculando el retraso y la multa. Reglas principales: máximo 3 préstamos activos por socio, solo pueden pedir préstamos los socios ACTIVOS, solo se presta si hay ejemplares (se descuenta uno al prestar y se repone al devolver), la fecha límite es de 7 días si no se indica y la multa es de $5 por día de retraso.

| Método | Ruta | Descripción |
|---|---|---|
| GET | `/api/prestamos` | Lista todos los préstamos |
| GET | `/api/prestamos/{id}` | Consulta un préstamo |
| POST | `/api/prestamos` | Registra un préstamo (JSON: isbn, socioId y fechaLimite opcional) |
| POST | `/api/prestamos/{id}/devolver` | Registra la devolución |
| DELETE | `/api/prestamos/{id}` | Elimina un préstamo (si estaba activo, repone el ejemplar) |

**Interfaz web.** Además de la API, el proyecto incluye una interfaz que usa los mismos servicios: `/vista/libros`, `/vista/usuarios` y `/vista/prestamos`.

## 3. Instrucciones para ejecutar localmente

### 3.1 Requisitos previos

- JDK 21
- Maven 3.9 o superior
- MySQL 8 en ejecución (puerto 3306 por defecto)

### 3.2 Pasos

1. **Obtener el proyecto.** Clona el repositorio indicado en la sección 1 y entra a la carpeta:

```
git clone https://github.com/Emma67g/LibreriaAPI.git
cd libreria
```

2. **Crear la base de datos.** Las tablas las crea la aplicación automáticamente al arrancar:

```
CREATE DATABASE libreria CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

3. **Configurar las credenciales de MySQL.** Por defecto se usan las definidas en `src/main/resources/application.properties` (usuario root). Si tu MySQL tiene otros datos, defínelos:

| Variable | Descripción | Valor por defecto |
|---|---|---|
| `SPRING_DATASOURCE_URL` | URL JDBC de la base de datos | jdbc:mysql://localhost:3306/libreria |
| `SPRING_DATASOURCE_USERNAME` | Usuario de MySQL | root |
| `SPRING_DATASOURCE_PASSWORD` | Contraseña de MySQL | Depende del usuario |
| `SERVER_PORT` | Puerto de la API | 1500 |

4. **Ejecutar la aplicación:**

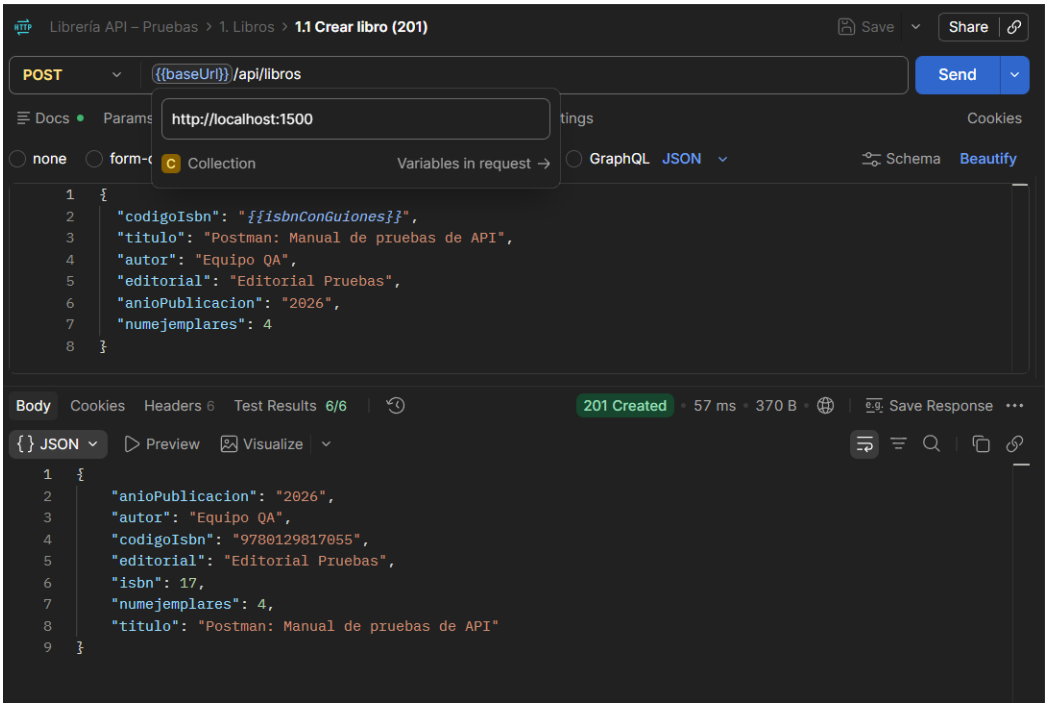
```
mvn spring-boot:run
```

5. **Abrir la aplicación.** Interfaz web en `http://localhost:1500` y API REST en `http://localhost:1500/api/libros`.

# 4. Capturas de pantalla de Postman

## 4.1 Libros

**Figura 1.** *Libros: registro de un libro (201 Created) y consulta (200 OK).*



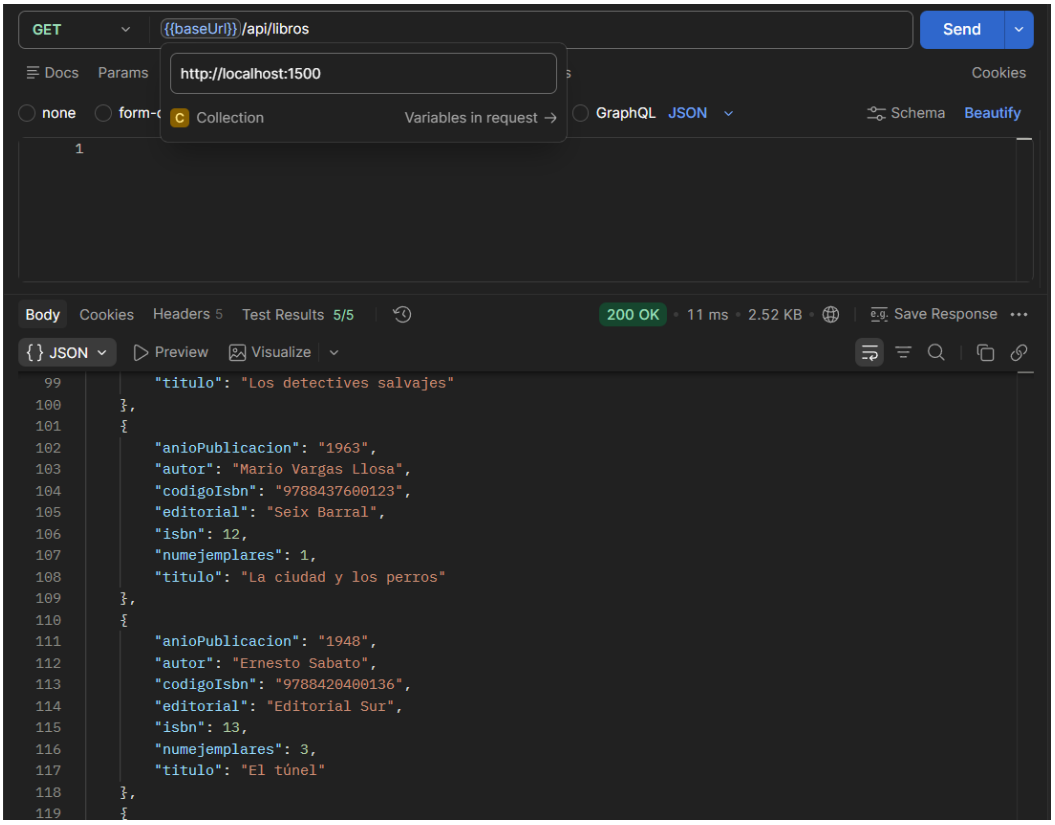
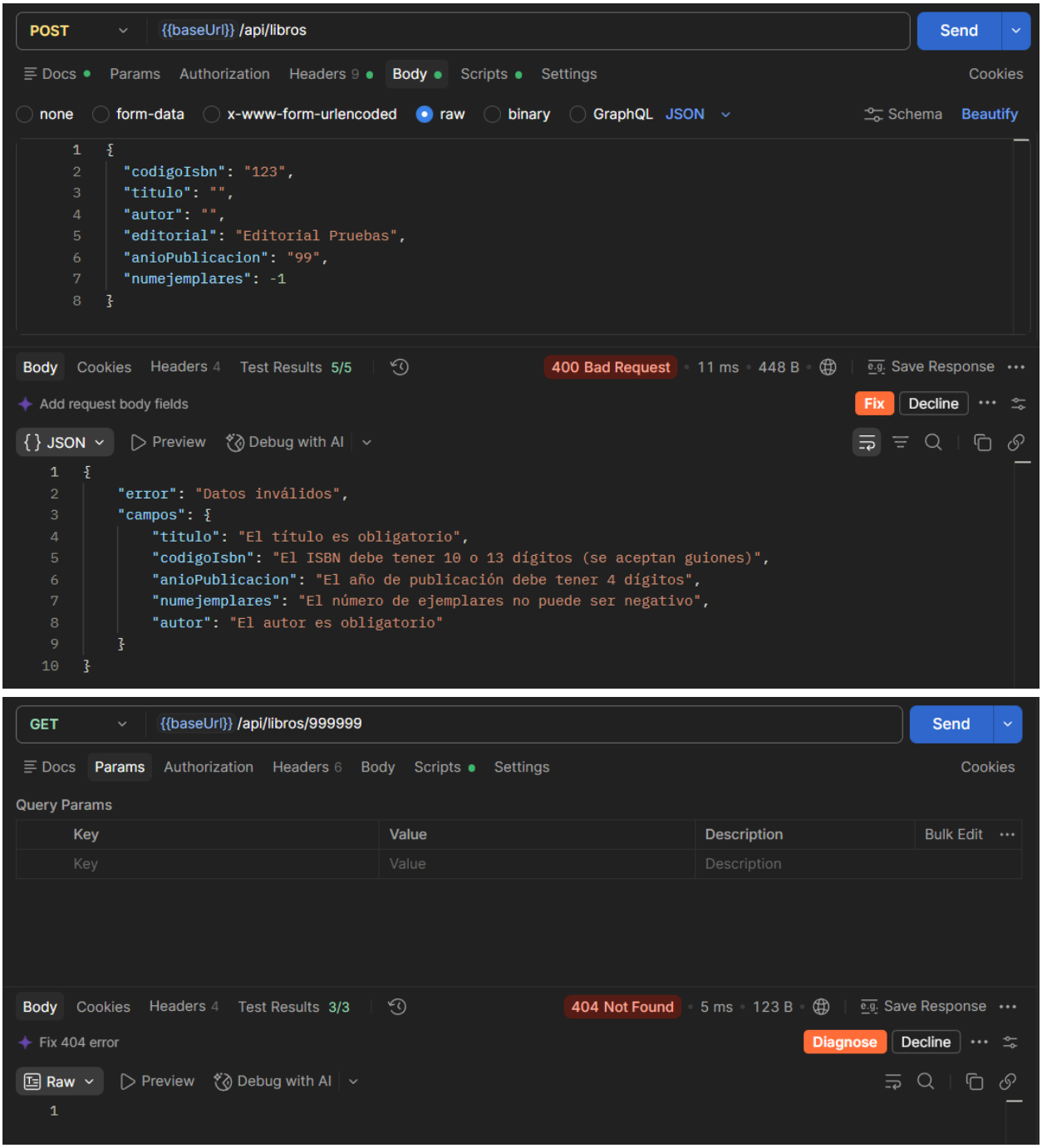
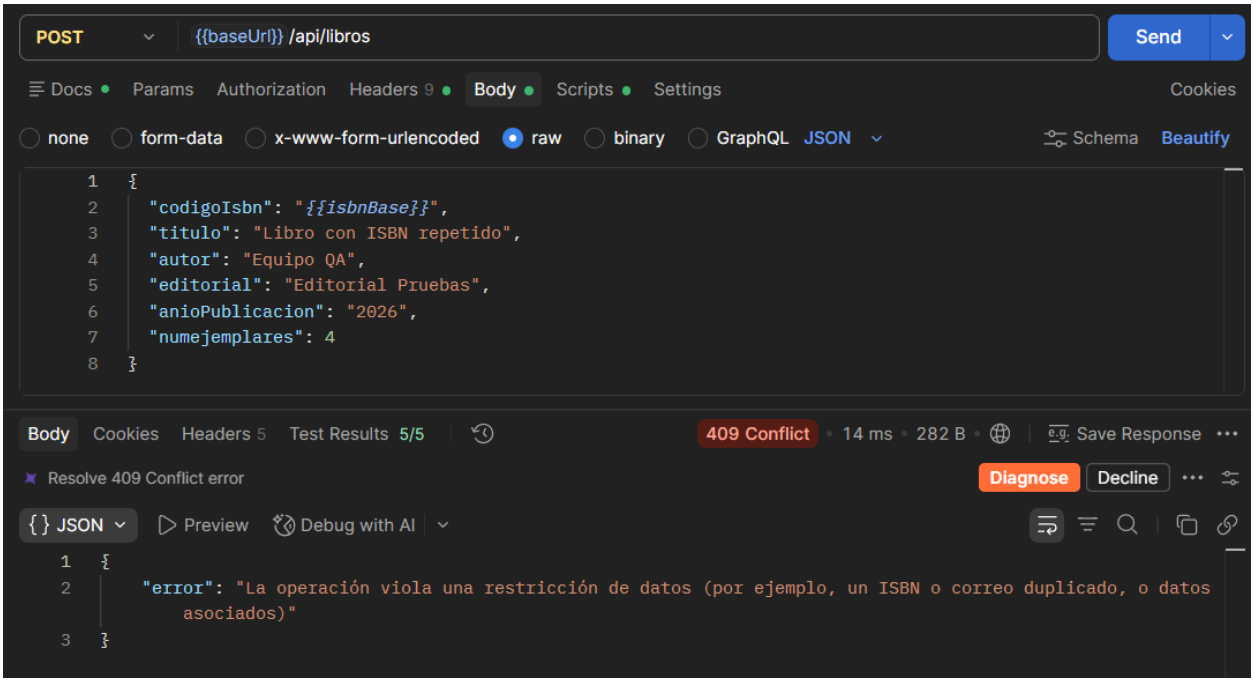

GET
{{baseUrl}}/api/libros
Send
Docs
Params
http://localhost:1500
Cookies
none
form-
Collection
Variables in request
GraphQL
JSON
Schema
Beautify
1
Body
Cookies
Headers 5
Test Results 5/5
200 OK
11 ms
2.52 KB
Save Response
{ } JSON
Preview
Visualize
"titulo": "Los detectives salvajes"
},
{
"anioPublicacion": "1963",
"autor": "Mario Vargas Llosa",
"codigoIsbn": "9788437600123",
"editorial": "Seix Barral",
"isbn": 12,
"numejemplares": 1,
"titulo": "La ciudad y los perros"
},
{
"anioPublicacion": "1948",
"autor": "Ernesto Sabato",
"codigoIsbn": "9788420400136",
"editorial": "Editorial Sur",
"isbn": 13,
"numejemplares": 3,
"titulo": "El túnel"
},
{

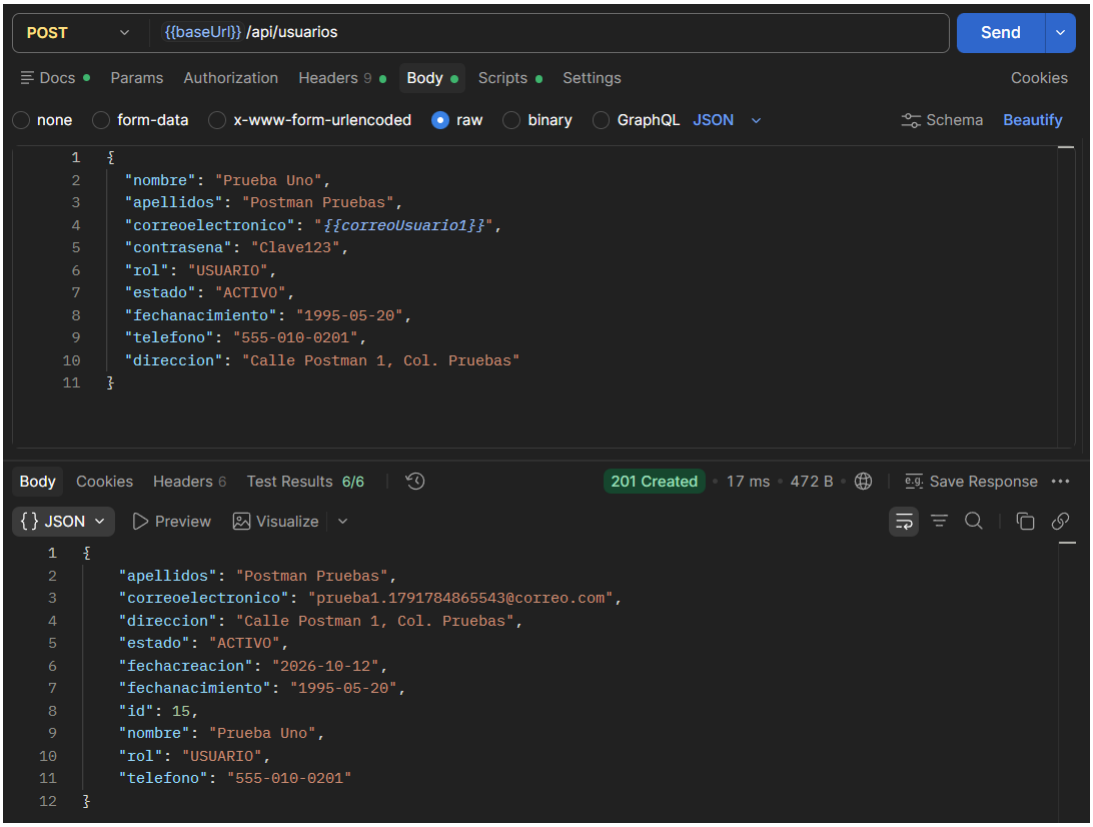
**Figura 2.** *Libros: errores de validación (400), libro inexistente (404) e ISBN duplicado (409).*






## 4.2 Usuarios

**Figura 3.** *Usuarios: alta y consulta; la respuesta no incluye la contraseña.*



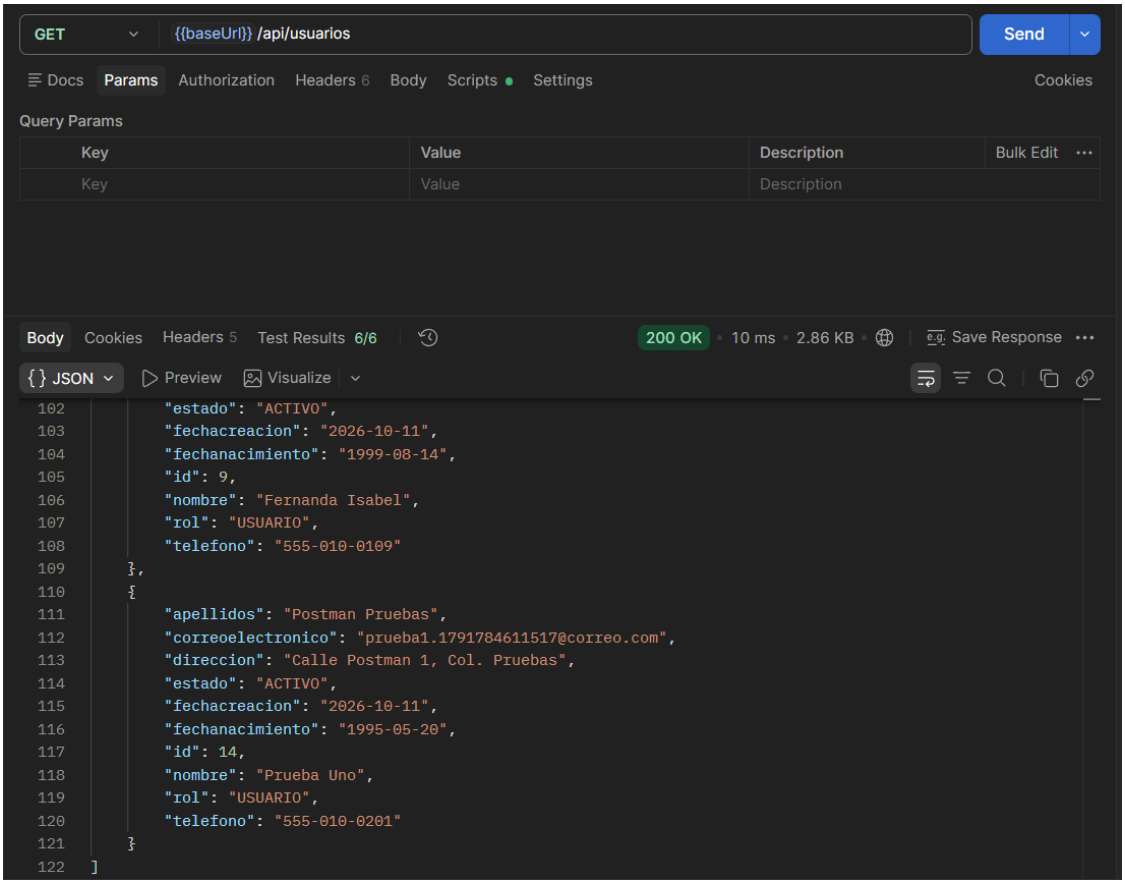
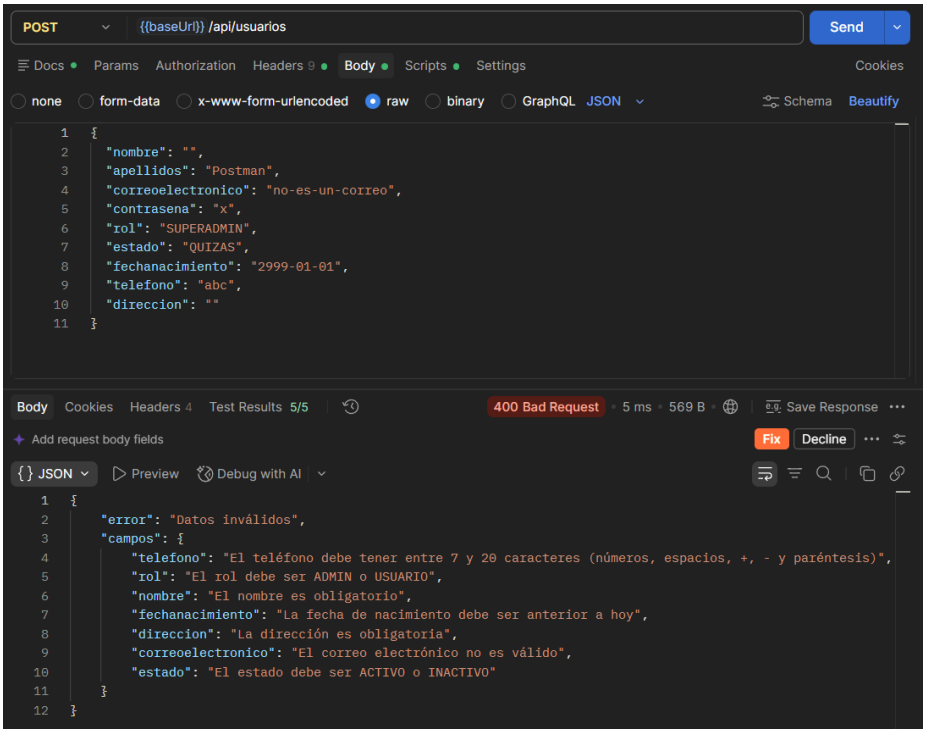
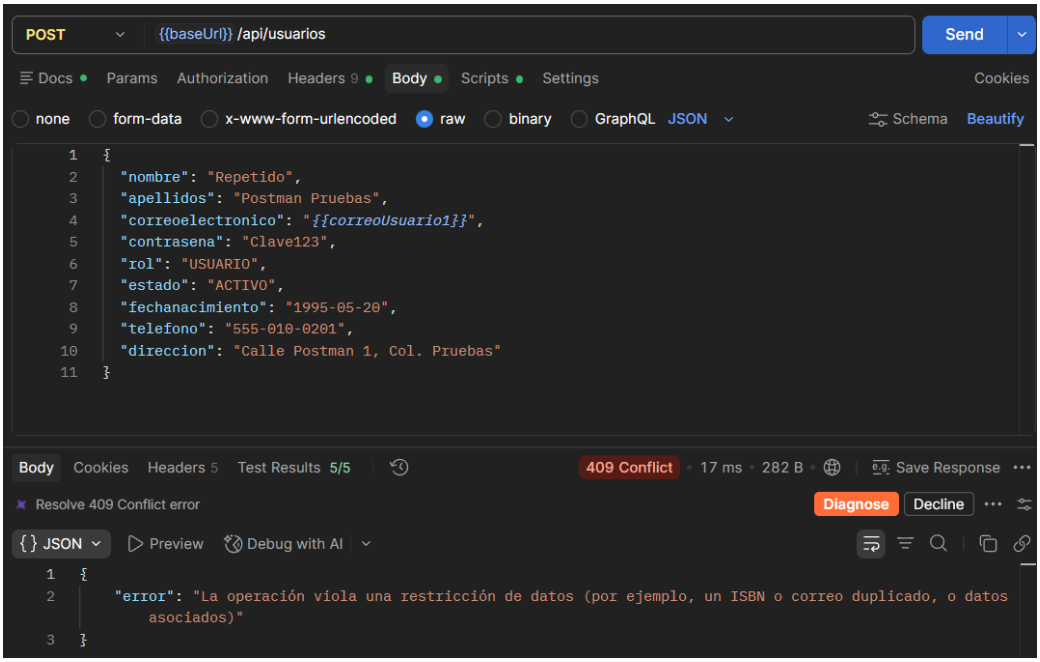


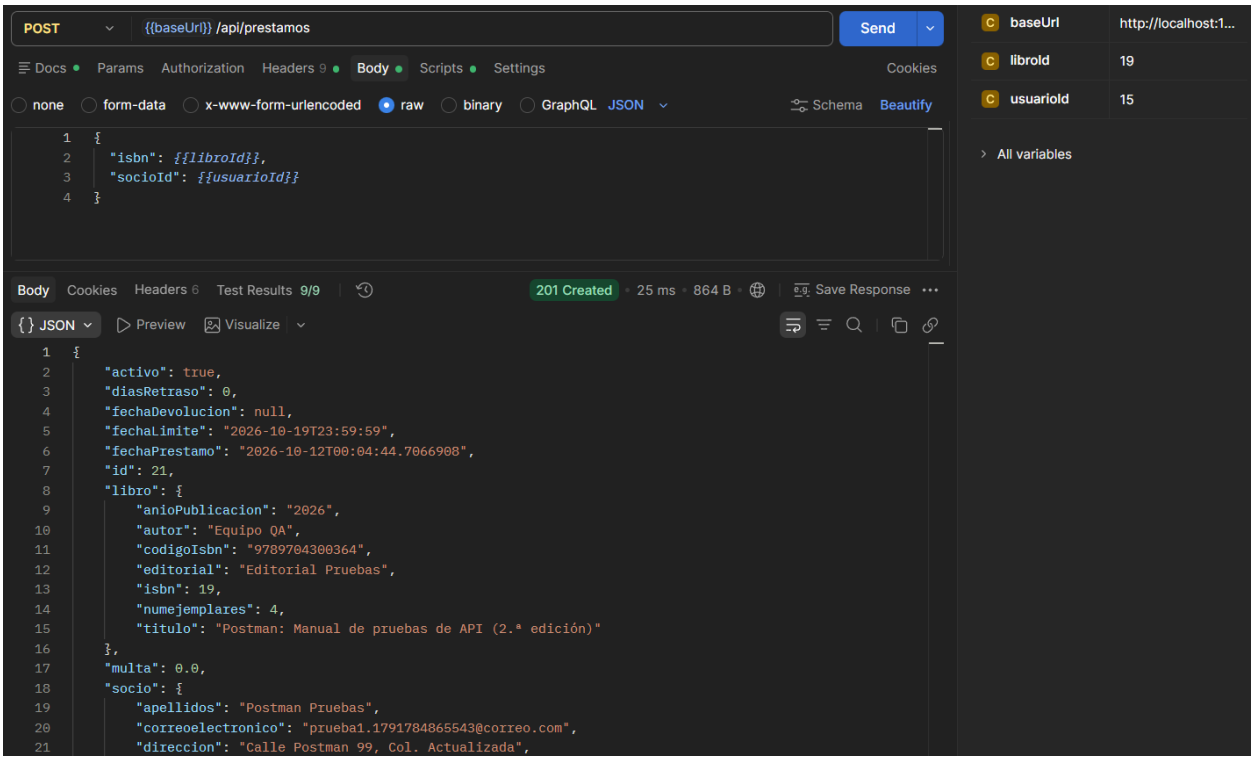

**Figura 4.** *Usuarios: errores de validación (400) y correo duplicado (409).*






## 4.3 Préstamos

**Figura 5.** *Préstamos: registro de un préstamo (201 Created) y verificación del inventario.*



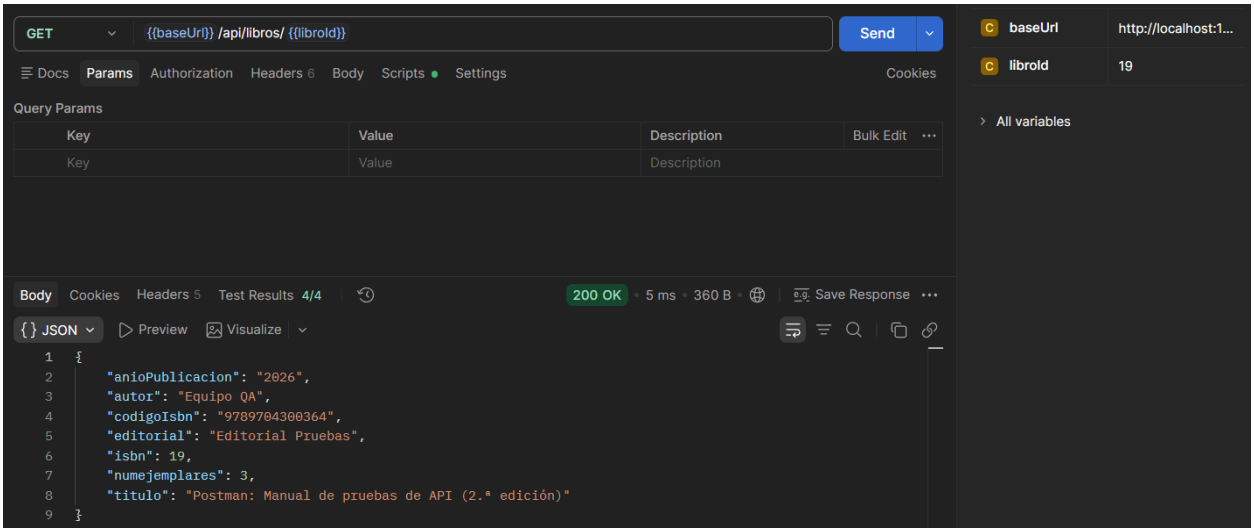


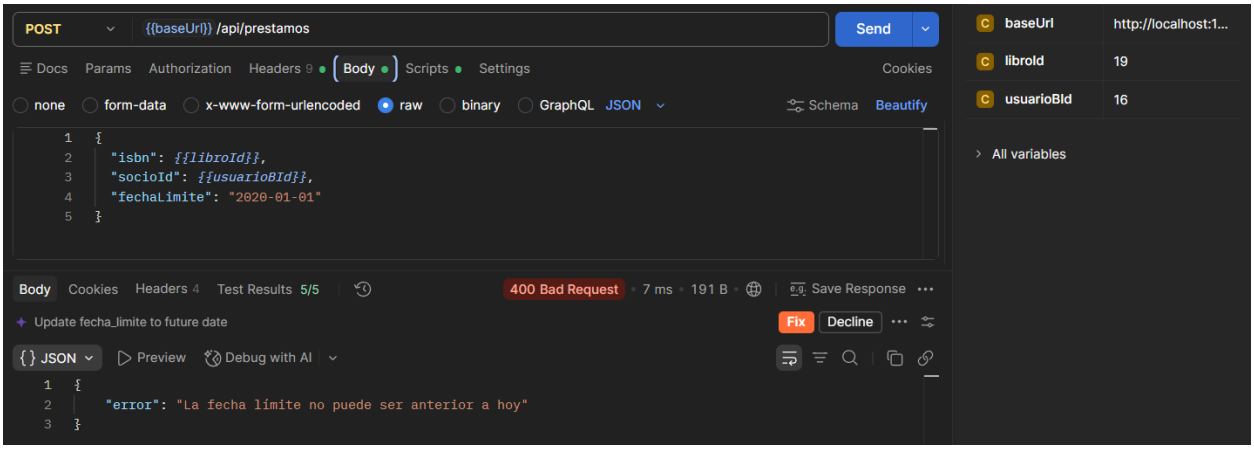

**Figura 6.** *Préstamos: errores de reglas de negocio (límite de 3 préstamos, sin ejemplares, socio inactivo y fecha inválida).*












## 4.4 Integridad referencial y limpieza

**Figura 7.** *Eliminación de un libro o usuario con préstamos (409).*

